

WEEK 4

Name =

M Abhi Charan

VTU No =

VTU28517

Subject = Problem Solving and Testing using JAVA

Slot = S5

Session- 1:

Task-1:

You are given a string *s* of even length. Split this string into two halves of equal lengths, and let *a* be the first half and *b* be the second half. Two strings are **alike** if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'). Notice that *s* contains uppercase and lowercase letters. Return true* if *a* and *b* are ***alike**. Otherwise, return false.

Example 1:

Input: *s* =

"book" Output:

true

Explanation: *a* = "bo" and *b* = "ok". *a* has 1 vowel and *b* has 1 vowel.

Therefore, they are alike.

Example 2:

Input: *s* =

"textbook"

Output: false

Explanation: *a* = "text" and *b* = "book". *a* has 1 vowel whereas *b* has 2.

Therefore, they are not alike.

Notice that the vowel *o* is counted

twice.

PROGRAM

```
class Solution {  
    public boolean halvesAreAlike(String s) {  
  
        int count1 =  
        0; int  
        count2 = 0;
```

```
int mid = s.length() / 2;
```

```

    for (int i = 0; i < mid;
        i++) { if
        (isVowel(s.charAt(i))) {
            count1++;
        }
    }

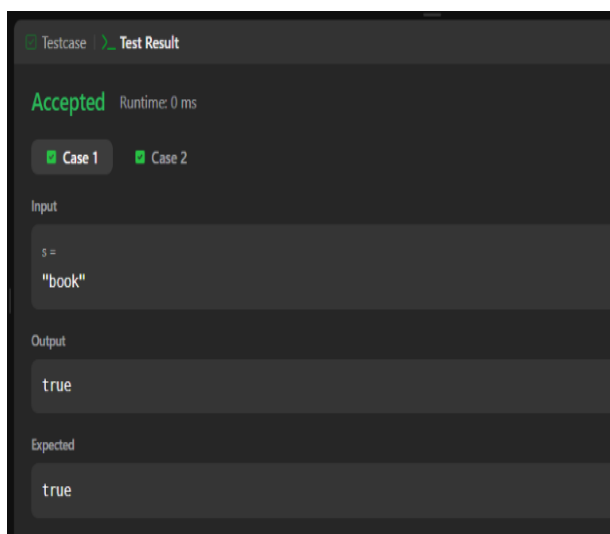
    for (int i = mid; i <
        s.length(); i++) { if
        (isVowel(s.charAt(i))) {
            count2++;
        }
    }

    return count1 == count2;
}

private boolean isVowel(char c) {
return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c
    == 'u'
|| c == 'A' || c == 'E' || c == 'I' || c == 'O' || c ==
    'U';
}

```

Output:



The screenshot shows a test result interface with a dark theme. At the top, there are tabs for 'Testcase' and 'Test Result'. Below the tabs, the status 'Accepted' is displayed in green, followed by 'Runtime: 0 ms'. Underneath, there are two buttons: 'Case 1' and 'Case 2', both with green checkmarks. The 'Input' section shows 's =' followed by a text box containing '"book"'. The 'Output' section shows a text box containing 'true'. The 'Expected' section shows a text box containing 'true'.

Task-2

Lapindromes

Lapindrome is defined as a string which when split in the middle, gives two halves having the same characters and same frequency of each character. If there are odd number of characters in the string, we ignore the middle character and check for lapindrome. For example ***gaga*** is a lapindrome, since the two halves ***ga*** and ***ga*** have the same characters with same frequency. Also, ***abccab***, ***rotor*** and ***xyzxy*** are a few examples of lapindromes.

Note that ***abbaab*** is NOT a lapindrome. The two halves contain the same characters but their frequencies do not match.

Your task is simple. Given a string, you need to tell if it is a lapindrome.

Input:

First line of input contains a single integer **T**, the number of test cases. Each test is a single line containing a string **S** composed of only lowercase English alphabet.

Output:

For each test case, output on a separate line: "YES" if the string is a lapindrome and "NO" if it is not.

Constraints:

- $1 \leq T \leq 100$
- $2 \leq |S| \leq 1000$, where $|S|$ denotes the

length of **S**

- **Sample 1:**

Input

Output

6

gaga

abccab

rotor

xyzxy

abbaab

ababc

EXPECTED OUTPUT

YE
S
NO
YE
S
YE
S
NO
NO

PROGRAM

import

java.util.*;

public class

Main {

public static void main(String[]

args) { Scanner sc = new

Scanner(System.in); int T =

sc.nextInt();

while (T-- > 0) {

String s = sc.next();

int[] count = new

int[26]; int n =

s.length();

int mid = n / 2;

// First half

for (int i = 0; i < mid; i++) {

count[s.charAt(i) - 'a']++;

```

    }

    // Second half
    int start = (n % 2 == 0) ? mid : mid + 1;

    for (int i = start; i < n; i++) {
        count[s.charAt(i) - 'a']--;
    }

    boolean lapindrome =
    true; for (int i = 0; i <
    26; i++) {
        if (count[i] != 0) {
            lapindrome =
            false; break;
        }
    }

    System.out.println(lapindrome ? "YES" : "NO");
    }

    sc.close();
}
}

```

Output:

Status : Successfully executed

Time:
0.0700 secs

Memory:
42.184 Mb

Sample Input

6
gaga
abcde
rotor
xyzxy
abbaab
ababc

Your Output

YES
NO
YES
YES
NO
NO

Session2:

Task-3

Alice and Bob each created one problem for HackerRank. A reviewer rates the two challenges, awarding points on a scale from 1 to 100 for three categories: *problem clarity*, *originality*, and *difficulty*.

The rating for Alice's challenge is the triplet $a = (a[0], a[1], a[2])$, and the rating for Bob's challenge is the triplet $b = (b[0], b[1], b[2])$.

The task is to calculate their comparison points by comparing each category:

- If $a[i] > b[i]$, then Alice is awarded 1 point.
- If $a[i] < b[i]$, then Bob is awarded 1 point.
- If $a[i] = b[i]$, then neither person receives a point.

Example

$a = [1, 2, 3]$

$b = [3, 2, 1]$

For elements $*0*$, Bob is awarded a point because $a[0] < b[0]$.

- For the equal elements $a[1]$ and $b[1]$, no points are earned.
- Finally, for elements 2, $a[2] > b[2]$ so Alice

receives a point. The return array is $[1, 1]$ with Alice's score first and Bob's second. **Function**

- **Description**

Complete the function *compareTriplets* with the following parameter(s):

- *int a[3]*: Alice's challenge rating
- *int b[3]*: Bob's challenge rating

Returns

- *int[2]*: the first element is Alice's score and the second is Bob's score

Input Format

The first line contains 3 space-separated integers, $a[0]$, $a[1]$, and $a[2]$, the respective values in triplet a .

The second line contains 3 space-separated integers, $b[0]$, $b[1]$, and $b[2]$, the respective values in triplet b .

Constraints

- $1 \leq a[i] \leq 100$
- $1 \leq b[i] \leq 100$

Sample Input 0

5 6 7

3 6 10

Sample Output 0

1 1

Explanation 0

In this example:

-
-

Now, let's compare each individual score:

- , so Alice receives point.
- , so nobody receives a point.
- , so Bob receives point.

Alice's comparison score is , and Bob's comparison score is . Thus, we return the array .

Sample Input 1

17 28 30

99 16 8

Sample Output 1

2 1

Explanation 1

Comparing the elements, so Bob receives a point.

Comparing the and elements, and so Alice receives two points. The return array is .

PROGRAM

```
import java.io.*;
import
java.math.*;
import
java.security.*;
import
java.text.*;
import java.util.*;
import java.util.concurrent.*;
```

```

import
java.util.function.*;
import
java.util.regex.*;
import
java.util.stream.*;
import static
java.util.stream.Collectors.joining;
import static
java.util.stream.Collectors.toList;

```

```

class Result {

```

```

    /*
     * Complete the 'compareTriplets' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     * 1. INTEGER_ARRAY a
     * 2. INTEGER_ARRAY b
     */

```

```

    public static List<Integer> compareTriplets(List<Integer> a,
        List<Integer> b) { int alice = 0;
        int bob = 0;

```

```

        for (int i = 0; i < 3;
            i++) { if (a.get(i)
                > b.get(i)) {
                    alice++;
                } else if (a.get(i) <
                    b.get(i)) { bob++;
                }
            }
        }

```

```

        List<Integer> result = new

```

```

        ArrayList<>(); result.add(alice);
        result.add(bob);

```

```

        b); return
    }
}

```

```
result;
```

```
}
```

```
}
```

```
public class Solution {  
    public static void main(String[] args) throws IOException {  
        BufferedReader bufferedReader = new  
            BufferedReader(new  
InputStreamReader(System.in));  
        BufferedWriter bufferedWriter = new BufferedWriter(new  
FileWriter(System.getenv("OUTPUT_PATH")));
```

```
List<Integer> a = Stream.of(bufferedReader.readLine().replaceAll("\\s+$",  
"").split(" "))  
    .map(Integer::parseInt)  
    .collect(toList());
```

```
List<Integer> b =  
Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))  
    .map(Integer::parseInt)  
    .collect(toList());
```

```
List<Integer> result = Result.compareTriplets(a,
```

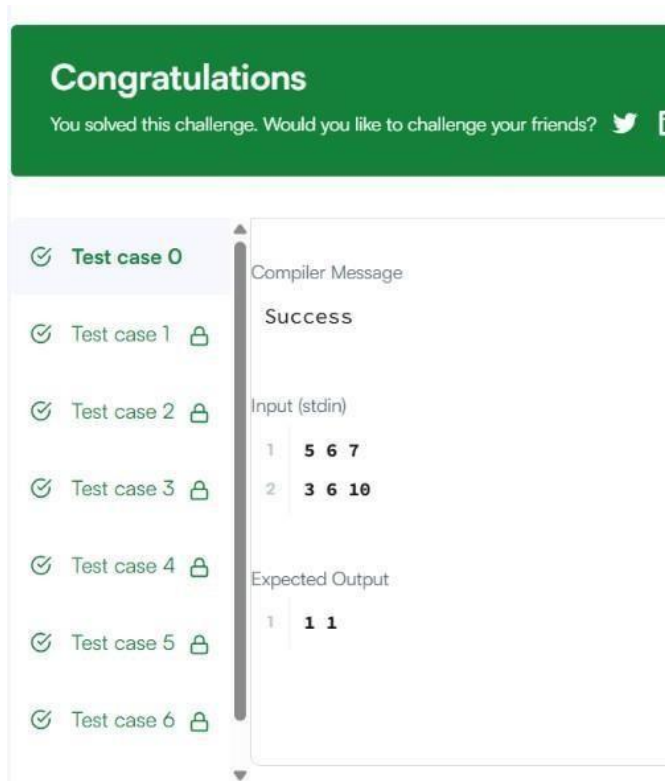
```
b); bufferedWriter.write(  
    result.stream()  
        .map(Object::toString)  
        .collect(joining(" "))  
    + "\n"  
);
```

```
bufferedReader.close();  
bufferedWriter.close();
```

```
}
```

```
}
```

OUTPUT:



Task4:

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums =`

`[1,2,3,1]` **Output:**

`true` **Explanation:**

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums =`

`[1,2,3,4]` **Output:**

`false` **Explanation:**

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

Constraints:

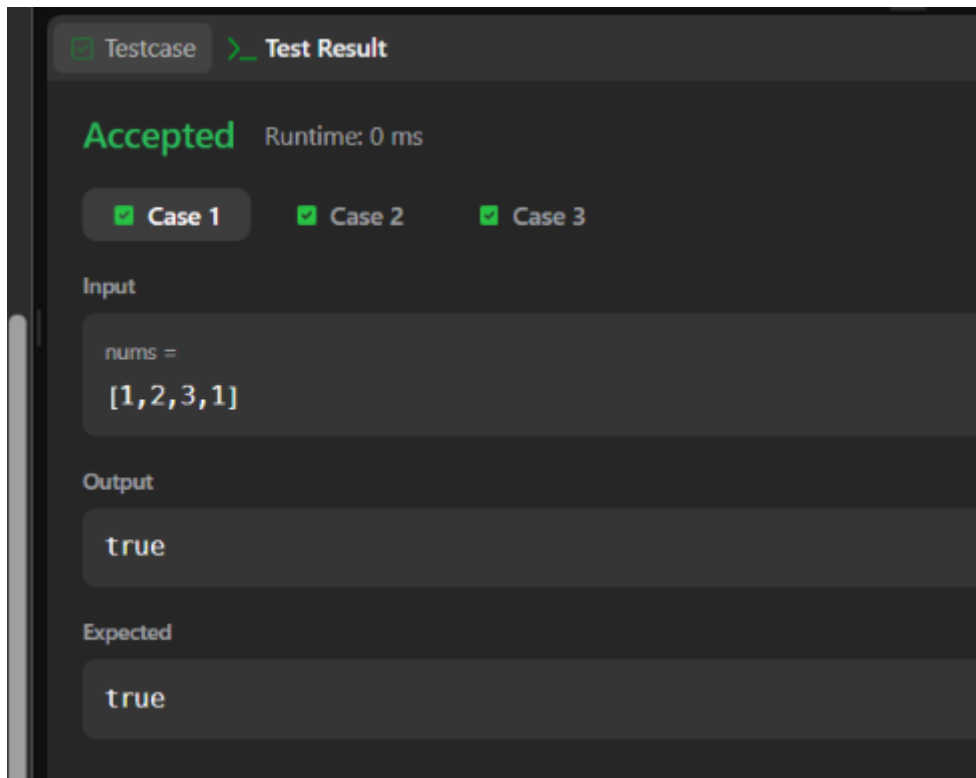
- $1 \leq \text{nums.length} \leq 10^5$
 $-10^9 \leq \text{nums}[i] \leq 10^9$

- **PROGRAM:**

```
import java.util.HashSet;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new
        HashSet<>(); for (int num : nums)
        {
            if
            (set.contains(n
            um)) { return
            true;
            }

            set.add(num);
        }
        return false;
    }
}
```



Session 3

Task5

Given a time in 12 [-hour AM/PM format](#), convert it to military (24-hour) time.

Note: - 12:00:00AM on a 12-hour clock is 00:00:00 on a 24-hour clock.

- 12:00:00PM on a 12-hour clock is 12:00:00 on a 24-hour clock.

Example

- Return '12:01:00'.
- Return '00:01:00'.

Function Description

Complete the function with the following parameter(s):

- : a time in hour format

Returns

- : the time in hour format

Input Format

A single string that represents a time in -hour clock format (i.e.: or).

Constraints

- All input times are valid

Sample Input 0

07:05:45PM

Sample Output 0

19:05:45

PROGRAM

```
import java.io.*;
import
java.math.*;
import
java.security.*;
import
java.text.*;
import java.util.*;
import
java.util.concurrent.*;
import
java.util.function.*;
import
java.util.regex.*;
import
java.util.stream.*;
import static
java.util.stream.Collectors.joining;
import static
java.util.stream.Collectors.toList;

class Result {

    /*
     * Complete the 'timeConversion' function below.
     *
     * The function is expected to return a STRING.
     * The function accepts STRING s as parameter.
     */

    public static String timeConversion(String s) {
        // Write your code here

        String period = s.substring(8);
        int hour = Integer.parseInt(s.substring(0, 2));

        if
            (period.equals("A
```

```

        M")) { if (hour ==
        12) {
            hour = 0;
        }
    } else {
        if (hour !=
            12) { hour
            += 12;
        }
    }
}

return String.format("%02d%s", hour, s.substring(2, 8));

}

}

public class Solution {
    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String s =

        bufferedReader.readLine(); String

        result =

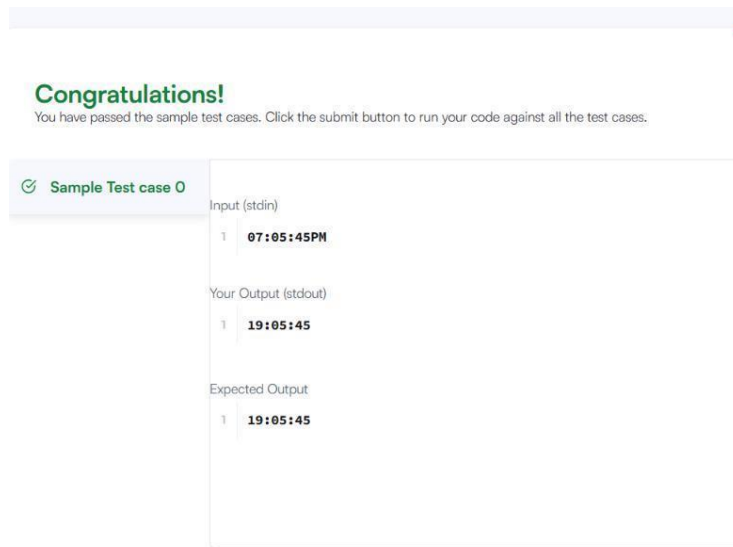
        Result.timeConversion(s);

        bufferedWriter.write(result);
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

Output:



Task-6

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.
Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`

Output: `[1,3,12,0,0]`

Example 2:

Input: `nums`

`= [0]` **Output:**

`[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

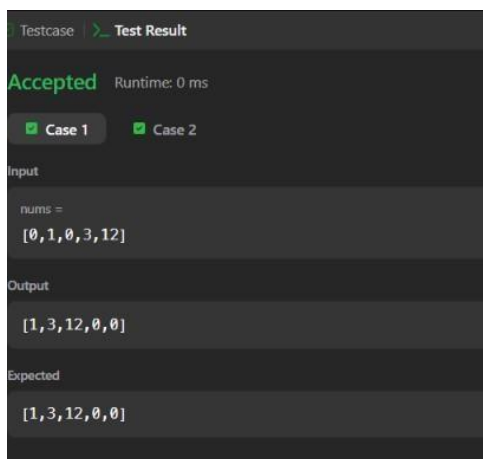
PROGRAM:

```
class Solution {
    public void moveZeroes(int[]

        nums) { int index = 0;

        // Move all non-zero elements to
        the front for (int i = 0; i <
        nums.length; i++) {
            if (nums[i] != 0) {
                nums[index] =
                nums[i]; index++;
            }
        }

        // Fill remaining positions
        with zero while (index <
        nums.length) {
            nums[index] = 0;
            index++;
        }
    }
}
```



```
}
```

Session 4

Task-7

Given a square matrix, calculate the absolute difference between the sums of its diagonals. For example:

1 2 3
4 5 6
9 8 9

The left-to-right diagonal sum is:

$$1 + 5 + 9 = 15$$

The right-to-left diagonal sum is:

$$3 + 5 + 9 = 17$$

Therefore, the absolute difference is:

$$|15 - 17| = 2$$

Program

:

```
import
java.io.*;
import
java.math.*;
import
java.security.*;
import
java.text.*;
import java.util.*;
import
java.util.concurrent.*;
import
java.util.function.*;
import
java.util.regex.*;
import
java.util.stream.*;
import static
java.util.stream.Collectors.joining;
import static
java.util.stream.Collectors.toList;
```

```

class Result {

    /*
     * Complete the 'diagonalDifference' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts 2D_INTEGER_ARRAY arr as parameter.
     */

    public static int diagonalDifference(List<List<Integer>> arr) {
        // Write your code

        here int n =

        arr.size();

        int primary =
        0; int
        secondary =
        0;

        for (int i = 0; i < n; i++) {

            primary += arr.get(i).get(i);
            secondary += arr.get(i).get(n
            - 1 - i);
        }

        return Math.abs(primary - secondary);

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new
        BufferedReader(new
        InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));
    }
}

```

```
int n = Integer.parseInt(bufferedReader.readLine().trim());  
List<List<Integer>> arr = new
```

Output:

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

✔ Test case 0

✔ Test case 1

✔ Test case 2

✔ Test case 3

✔ Test case 4

✔ Test case 5

✔ Test case 6

Compiler Message

Success

Input (stdin)

1 3

2 11 2 4

3 4 5 6

4 10 8 -12

Expected Output

1 15

Task-8

Given a 2D integer array matrix, return the transpose of matrix

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [[1,4,7],[2,5,8],[3,6,9]]

Example 2:

Input: matrix = [[1,2,3],[4,5,6]]

Output: [[1,4],[2,5],[3,6]]

Program:

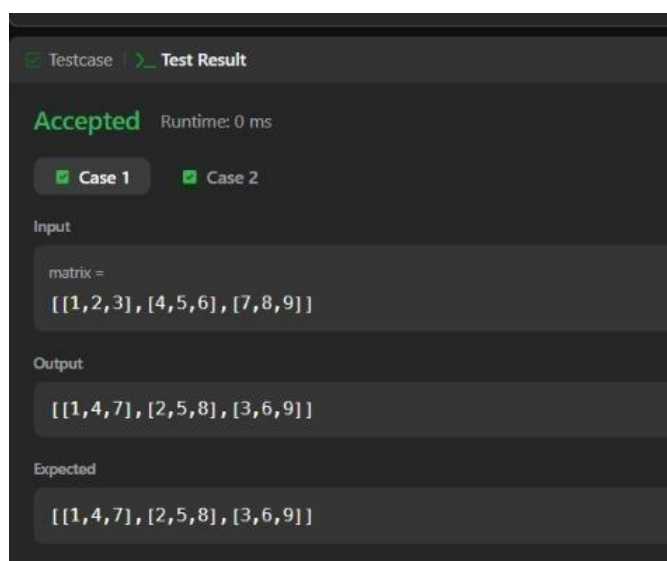
```
Class Solution {
    Public int[][] transpose(int[][]
        matrix) {    Int  rows  =
        matrix.length;
        Int cols = matrix[0].length;
        // Transposed matrix: cols
        × rows Int[][] result = new
        int[cols][rows]; For (int l =
        0; l < rows; i++) {
```

```

        For (int j = 0; j < cols;
            j++) { Result[j][i] =
                matrix[i][j];
            }
        }
    Return result;
}
}

```

Output:



Session 5 Task-9

Given a $m \times n$ matrix mat and an integer k , return a matrix $answer$ where each $answer[i][j]$ is the sum of all elements $mat[r][c]$ for:

$i - k \leq r \leq i + k$,

$j - k \leq c \leq j + k$, and

(r, c) is a valid position in the

matrix. Example 1:

Input: $mat = [[1,2,3],[4,5,6],[7,8,9]]$, $k = 1$

Output: $[[12,21,16],[27,45,33],[24,39,28]]$

Example 2:

Input: $mat = [[1,2,3],[4,5,6],[7,8,9]]$, $k = 2$

Output: $[[45,45,45],[45,45,45],[45,45,45]]$

Program:

Class Solution {

```

Public int[][] matrixBlockSum(int[][] mat, int k) {

    Int m =
    mat.length; Int n
    = mat[0].length;

    Int[][] ans = new
    int[m][n]; For (int l =
    0; l < m; l++) {
        For (int j = 0; j < n;
        j++) { Int sum =
        0;
        Int rowStart = Math.max(0, l
        - k); Int rowEnd =
        Math.min(m - 1, l + k); Int
        colStart = Math.max(0, j -
        k);
        Int colEnd = Math.min(n - 1, j + k);
        For (int r = rowStart; r <= rowEnd;
        r++) { For (int c = colStart; c <=
        colEnd; c++) {
            Sum += mat[r][c];
        }
        }
        Ans[l][j] = sum;
    }
    }
    Return ans;
}
}

```

Output:

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

mat =
[[1,2,3],[4,5,6],[7,8,9]]

k =
1

Output

[[12,21,16],[27,45,33],[24,39,28]]

Expected

[[12,21,16],[27,45,33],[24,39,28]]

Task-10

You are given a 2D matrix of dimension $m \times n$ and a positive integer k . You have to rotate the matrix k times and print the resultant matrix. Rotation should be in anti-clockwise direction.

Rotation of a $m \times n$ matrix is represented by the following figure. Note that in one rotation, you have to shift elements by one step only.

[matrix-rotation](https://hr-challenge-

images.s3.amazonaws.com/2517/matrix-rotation.png) It is guaranteed

that the minimum of m and n will be even.

As an example rotate the Start matrix by 2:

```
..
`
Start   First   Second
1 2 3 4  2 3 4 5  3 4 5 6
12 1 2 -> 1 2 3 6  2 3 4 7
   5          ->

11 4 3   12 1 4 7  1 2 1 8
   6
10 9 8   11 10 9 8  12 11 10
   7          9
..
`
```

****Function Description****

Complete the `matrixRotation` function in the editor below. `matrixRotation` has the following

parameter(s):

- `int matrix[m][n]:` a 2D array of integers
- `int r:` the rotation factor

****Prints****

It should print the resultant 2D integer array and return nothing. Print each row on a separate line as space-separated integers.

****Input Format****

The first line contains three space separated integers, `svg`, `svg`, and `svg`, the number of rows and columns in `svg`, and the required rotation.
The next `svg` lines contain `svg` space-separated integers representing the elements of a row of `svg`.

****Constraints****

* `svg`
`svg`
`sv`
`g`
`sv`
`g`

****Sample Input****

****Sample Input #01****

...

STDIN Function

4 4 2 rows m = 4, columns n = 4, rotation factor r = 2
1 2 3 4 matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]]
5 6 7 8
9 10 11 12
13 14 15 16

...

****Sample Output #01****

...

3 4 8 12
2 11 10 16
1 7 6 15
5 9 13 14

****Explanation #01****

The matrix is rotated through two rotations.

..

```
` 1 2 3 4 2 3 4 8 3 4 8 12
   5 6 7 8 1 7 11 12 2 11 10 16
   9 10 11 12 -> 5 6 10 16 -> 1 7 6 15
  13 14 15 16 9 13 14 15 5 9 13 14
```

...

****Sample Input #02****

...

```
5 4 7
1 2 3 4
7 8 9 10
13 14 15 16
19 20 21 22
25 26 27 28
```

...

****Sample Output #02****

...

```
28 27 26 25
22 9 15 19
```

16 8 21 13
10 14 20 7
4 3 2 1

...

****Explanation 02****

The various states through 7 rotations:

..

` 1 2 3 4 2 3 4 10 3 4 10 16 4 10 16 22
7 8 9 10 1 9 15 16 2 15 21 22 3 21 20 28
13 14 15 16 -> 7 8 21 22 -> 1 9 20 28 -> 2 15 14 27 ->
19 20 21 13 14 20 7 8 14 1 9 8 26
22 28 27
25 26 27 19 25 26 13 19 25 7 13 19
28 27 26 25

10 16 22 16 22 28 22 28 27 28 27 26
28 27 26 25
4 20 14 10 14 8 16 8 9 25 22 9 15 19
27 26
3 21 8 26 -> 4 20 9 25 -> 10 14 15 19 -> 16 8 21 13
2 15 9 25 3 21 15 19 4 20 21 13 10 14 20 7
1 7 13 19 2 1 7 13 3 2 1 7 4 3 2 1

****Sample Input #03****

...

2 2 3
1 1
1 1

****Sample Output #03****

...

1
1
1 1

****Explanation #03****

All of the elements are the same, so any rotation will repeat the same matrix.

Program

```

import
java.io.*;
import
java.math.*;
import
java.security.*;
import
java.text.*;
import java.util.*;
import
java.util.concurrent.*;
import
java.util.function.*;
import
java.util.regex.*;
import
java.util.stream.*;
import static
java.util.stream.Collectors.joining;
import static
java.util.stream.Collectors.toList;

class Result {

public static void matrixRotation(List<List<Integer>>

    matrix, int r) { int m = matrix.size();
    int n = matrix.get(0).size();

    int layers = Math.min(m, n) / 2;

    for (int layer = 0; layer < layers;

        layer++) { List<Integer> elements =

            new ArrayList<>();

            int top =
            layer; int left
            = layer;
            int bottom = m - 1 -

```

```
layer; int right = n - 1  
- layer;
```

```

// Top row: left -> right
for (int j = left; j <= right; j++) {
    elements.add(matrix.get(top).get(j));
}

// Right column: top+1 -> bottom
for (int i = top + 1; i <= bottom; i++) {
    elements.add(matrix.get(i).get(right));
}

// Bottom row: right-1 -> left
for (int j = right - 1; j >= left; j--) {
    elements.add(matrix.get(bottom).get(j));
}

// Left column: bottom-1 -> top+1
for (int i = bottom - 1; i > top; i--) {
    elements.add(matrix.get(i).get(left));
}

// Rotate anti-clockwise
int rotation = r % elements.size();

Collections.rotate(elements, -rotation);
int index = 0;

// Top row
for (int j = left; j <= right; j++) {

```

```

        matrix.get(top).set(j, elements.get(index++));
    }

    // Right column
    for (int i = top + 1; i <= bottom; i++) {
        matrix.get(i).set(right,
            elements.get(index++));
    }

    // Bottom row
    for (int j = right - 1; j >= left; j--) {
        matrix.get(bottom).set(j,
            elements.get(index++));
    }

    // Left column
    for (int i = bottom - 1; i > top; i--) {
        matrix.get(i).set(left,
            elements.get(index++));
    }
}

// Print the rotated
matrix for (int i = 0; i
< m; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(matrix.get(i).ge
            t(j));

        if (j < n - 1) {
            System.out.print(
                " ");
        }
    }
    System.out.println();
}
}
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new

```

```

        BufferedReader(new
InputStreamReader(System.in));

        String[] firstMultipleInput =

        bufferedReader.readLine().replaceAll("\\s+$", "").split(" "); int m =

        Integer.parseInt(firstMultipleInput[0]);

        int n =

        Integer.parseInt(firstMultipleInput[1]

        ); int r =

        Integer.parseInt(firstMultipleInput[2]

        );

        List<List<Integer>> matrix = new ArrayList<>();

        IntStream.range(0,
            m).forEach(i -> { try {
                matrix.add(
                    Stream.of(bufferedReader.readLine().replaceAll("\\s+$",
                        "").split(" "))
                        .map(Integer::parseInt)
                        .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        Result.matrixRotation(matrix, r);

        bufferedReader.close();
    }

```

Output:

Congratulations

You solved this challenge. Would you like to challenge your friends?

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

✓ Test case 6

Compiler Message

Success

Input (stdin)

1	4 4 1
2	1 2 3 4
3	5 6 7 8
4	9 10 11 12
5	13 14 15 16

Expected Output

1	2 3 4 8
---	---------